Тренажёр оператора ДДС города Москвы

Сопроводительная документация

Задача №9 Департамента по делам гражданской обороны, чрезвычайным ситуациям и пожарной безопасности города Москвы. Лидеры цифровой трансформации, 2026.

---

1. Назначение

Тренажёр готовит операторов дежурно-диспетчерских служб и Системы-112 к работе в боевой системе **ПОВ-112 (АРМ-112)**. Курсант принимает учебный вызов: разговаривает с заявителем, заполняет карточку происшествия, классифицирует происшествие и оповещает службы — в интерфейсе, повторяющем боевой. После вызова система показывает разбор с ценой каждой ошибки.

Ключевая особенность решения — **тренажёр отличает выясненное от угаданного**. Раздел 3.3 объясняет, почему это главная метрика подготовки оператора.

---

2. Состав решения

Компонент	Технология	Назначение
Веб-клиент	React 19, TypeScript, Vite	интерфейс АРМ-112 и учебный слой
Сервер приложения	Rust, axum, tokio	API, домен, оценка, фоновые задачи
Хранилище	PostgreSQL 16	классификатор, вызовы, занятия, журналы
Языковая модель	GigaChat 3.1 10B-A1.8B на llama.cpp	реплики заявителя, подготовка вызовов, сводки

Все четыре компонента разворачиваются на одной машине без выхода в интернет и без графического ускорителя. Обоснование — раздел 6.

Объём исходного кода: 9 837 строк Rust в 51 файле, 3 995 строк TypeScript в 42 файлах, 17 миграций схемы базы.

---

3. Методы обработки данных

3.1. Классификация происшествия — детерминированная, без нейросети

Единый классификатор происшествий (ЕКП) — это таблица, где каждой комбинации признаков соответствует **ровно один** правильный ответ. Поэтому классификацию считает не модель, а прямой поиск по таблице:

```
ответы карты ──①──► признаки ──②──► позиция ЕКП ──③──► список оповещения
```

① Ответы опросной карты сопоставляются с признаками трёх уровней («где», «что именно», «уточнение»). ② По группе и признакам ищется позиция классификатора. Если незаполненный уровень признаков оставляет несколько позиций с разными исходами, система **не выбирает первую попавшуюся**, а называет варианты и просит уточнить — так же ведёт себя боевая система. ③ Позиция задаёт список оповещения. Службы, получающие лишь копию карточки (значения «карточка-112», «нет реагирования»), отделяются от реагирующих: оценка не должна штрафовать за то, что оператор их не вызвал.

В активной версии классификатора **1 283 позиции в 24 группах**, **134 593 строки матрицы оповещения**, 60 служб.

Такой выбор даёт воспроизводимость: эталон для учебного вызова не зависит от того, какая модель загружена и загружена ли вообще.

3.2. Языковая модель — только текст

Модель отвечает за то, что нельзя посчитать: реплики заявителя в разговоре, черновики учебных вызовов, словесные сводки по группе. **Эталон для оценки она не считает никогда** — его считает классификатор из раздела 3.1.

При подготовке учебного вызова конвейер устроен так:

1. признаки выбираются из **закрытого словаря** классификатора;
2. ответы, которых нет в словаре, отбрасываются;
3. классификатор считает эталон;
4. если он жалуется (например, «уточните признак второго уровня»), его жалоба возвращается модели как замечание, и шаг повторяется;
5. только после этого модель пишет текст легенды и скрытых фактов.

Ответ модели ограничен **схемой JSON на уровне декодирования** (`response_format: json_schema`): движок отбрасывает токены, нарушающие схему, поэтому структурно неверный ответ невозможен. До этого модель на десять миллиардов параметров регулярно клала значения не в те поля, как её ни просили в подсказке.

3.3. Оценка — три слоя

Разбор сравнивает три независимых слоя данных:

Слой	Что это
Скрытая истина	что на самом деле произошло, заложено в сценарий
Стенограмма	что заявитель действительно сообщил в разговоре
Карточка	что оператор ввёл в поля

Из сопоставления рождаются четыре вида замечаний с разным весом:

Вид	Вес	Смысл
`guessed`	1	значение верное, но заявитель его не сообщал — **угадал**
`extra`	1	вызвана лишняя служба
`missing`	2	служба не оповещена
`wrong`	3	значение не совпало с эталоном

Превышение норматива времени добавляет 2.

Замечание формулируется как последствие, а не как балл: не «−3 за поле „служба“», а «служба не оповещена — фактически не выехала».

Смысл метрики «угадал»: оператор, угадавший тип происшествия по первой фразе, в боевой обстановке ошибётся, как только вызов окажется нетипичным. На живых данных стенда доля угаданного составила **38 %** — то есть больше трети верных ответов курсанты не выясняли у заявителя.

3.4. Статусы карточки — вывод, а не переход

Статус карточки ДДС не хранится как состояние конечного автомата, а **выводится** из состояния служб и прошедшего времени. Это избавляет от рассинхронизации: любое изменение реагирования сразу даёт верный статус. Фоновая задача (тик 5 секунд) пересчитывает статусы и переводит карточки в «Не оповещено» и «Не завершено» при превышении нормативов; запись условная — по прежнему статусу, поэтому два экземпляра службы не могут записать переход дважды.

3.5. Исходные данные

Все данные, на которых работает тренажёр, взяты из материалов задачи. Их происхождение построчно зафиксировано в `PROVENANCE.md`:

- **ЕКП** — файлы классификатора двух версий (v046_11 от 15.11.2024 и v046_24 с корректировкой МВД и Департамента), импортированы целиком;
- **учебные вызовы** — 96 обращений, разобранных из 32 бумажных билетов; у всех 96 разобран телефон, у 57 — ФИО заявителя, у 25 — скрытый адрес;
- **интерфейс** — снят по скриншотам боевой системы, сопоставление кадров описано в `frontend/SCENARIOS.md`.

Синтетических данных, придуманных разработчиком, в поставке нет.

---

4. Функциональная архитектура

4.1. Роли

Роль	Доступ
Курсант	учебные вызовы, своя история, свой разбор
Преподаватель	подготовка вызовов, группы, аналитика по группе
Администратор	версии классификатора, учётные записи, системные журналы

Разграничение проверяется на сервере при каждом запросе; интерфейс скрывает недоступные разделы, но не является средством защиты.

4.2. Занятие курсанта

```
вход → журнал происшествий → учебный вызов
                                  |
              разговор с заявителем ⟷ заполнение карточки 112
                                  |
                        классификация → список оповещения
                                  |
                         карточка ДДС: статусы реагирования
                                  |
                               разбор с оценкой
```

Ввод в карточку отправляется на сервер пакетами с задержкой; правки в пределах одного окна **сливаются вглубь**, а не затирают друг друга. При обрыве сети пакет повторяется; при отказе сервера правка снимается, а не копится молча.

4.3. Подготовка вызовов преподавателем

Преподаватель ведёт с моделью переписку об учебном вызове: «поменяй ФИО», «добавь вопрос про детей», «сделай первую фразу короче». Модель отвечает репликой и патчем; патч применяется к вызову, эталон пересчитывается классификатором. Переписка сохраняется между заходами.

Готовый вызов преподаватель утверждает — до утверждения курсанты его не видят.

4.4. Аналитика

Диаграммы (тепловая карта ошибок, динамика, воронка, виды замечаний) строятся из настоящей статистики занятий средствами SVG, без сторонних библиотек. Словесную сводку по группе формулирует модель — но **числа в неё подаются уже посчитанными**, в виде именованных агрегатов.

---

5. Компонентная архитектура

5.1. Сервер

```
api/        HTTP-обработчики, схема OpenAPI
domain/     классификатор, статусы, оценка, подготовка вызовов
ingest/     импорт классификатора и билетов
jobs/       фоновые задачи: сторож нормативов, очередь заданий модели
ml/         адаптеры моделей: HTTP-сервер модели и заглушка
```

Доменный слой не знает ни о HTTP, ни о моделях: классификатор и оценка — чистые функции над данными. Поэтому 138 серверных проверок выполняются без поднятия модели.

Описание API машиночитаемо (OpenAPI, utoipa). Отдельная проверка читает список маршрутов из исходного кода и **падает при расхождении** со схемой — однажды это уже поймало четыре незадокументированных маршрута.

5.2. Клиент

Слоистая раскладка (Feature-Sliced Design): `shared` → `entities` → `widgets` → `pages`. Состояние — zustand, стили — модули SCSS, маршрутизация — HashRouter (стенд отдаётся статикой без настройки переписывания путей).

Канал событий WebSocket с переподключением по возрастающей задержке. Канал — **ускорение, а не источник истины**: при его отказе интерфейс продолжает работать на обычных запросах.

5.3. Сервер модели

llama.cpp в режиме сервера, совместимого с OpenAI API. Подробности — `backend/docs/MODEL-SERVER.md`.

---

6. Условия и ограничения

6.1. Целевой стенд

Процессор Intel Core i7 или Xeon, не менее 6 ядер; 32 ГБ оперативной памяти; **графический ускоритель не предусмотрен**. Это задано системными требованиями, и из этого следует весь выбор в разделе 3.

6.2. Что из этого следует

Модель на процессоре — узкое место. Выбрана **GigaChat 3.1 10B-A1.8B**: из десяти миллиардов параметров на каждом токене работают 1,8 миллиарда (архитектура со смесью экспертов, 64 эксперта, используются 4). Скорость на процессоре определяется активными параметрами, а не общим размером, поэтому модель отвечает как небольшая, а знает как крупная. Лицензия MIT.

Дополнительно включено **спекулятивное декодирование на блоке MTP** (multi-token prediction), встроенном в модель. Поддержки этого режима для данной архитектуры в llama.cpp не было — она добавлена двумя патчами, которые приложены к поставке (`backend/docs/mtp-*.patch`).

Измеренный результат на 6 потоках процессора: **10,3 → 14,8 токенов в секунду, прирост 44 %**. Существенно, что `--spec-draft-n-max 1`: при значении по умолчанию 3 проверка отбрасывала большую часть черновика (доля принятых токенов 30 %), при 1 — принимается 73 %.

Оговорка к замеру: сравнивались два прогона на одной машине при одинаковой нагрузке, а не усреднение серии. Порядок величины достоверен, вторая значащая цифра — нет.

6.3. Ограничения решения

- **Распознавание и синтез речи не входят в поставку.** Разговор с заявителем ведётся текстом. Точки подключения адаптеров заложены (`ml/`), но модели не выбраны.
- **Сводка по группе считается около минуты** — числа берёт база, но формулирует их модель на процессоре.
- **Тренажёр не подключается к боевой ПОВ-112.** Он воспроизводит её интерфейс и логику по материалам задачи; обмена данными с действующей системой нет и не предполагается.
- **Часть решений принята при неполных исходных данных.** Все такие места перечислены в `OPEN-QUESTIONS.md` с указанием, что именно неизвестно и какое допущение сделано, — а не замаскированы.

---

7. Сборка, установка и запуск

7.1. Что должно быть на машине

Компонент	Версия	Проверено на
Rust	см. примечание	1.98.1
Node.js	20.19+	20.20.0
PostgreSQL	14+	16 (в контейнере)
Docker	любая	для базы; необязателен

Нижняя граница версии Rust не проверялась: указана та, на которой решение собрано. Редакция языка — 2021.

7.2. База данных

```
docker run -d --name arm112-db \
  -e POSTGRES_USER=arm112 -e POSTGRES_PASSWORD=arm112 \
  -e POSTGRES_DB=arm112 -p 5432:5432 postgres:16
```

Схема разворачивается миграциями при первом запуске сервера.

7.3. Сервер приложения

```
cd backend
cp .env.example .env          # при необходимости поправить DATABASE_URL
cargo build --release
cargo run --release           # http://localhost:8080
```

Описание API после запуска: `http://localhost:8080/api/docs`.

Наполнение данными (классификатор и учебные вызовы) выполняется отдельными командами импорта, см. `BACKEND.md`, раздел 14.

7.4. Веб-клиент

```
cd frontend
npm run install:all
npm run build                 # сборка в apps/react/dist
npm run dev                   # разработка, http://localhost:5173
```

Собранный клиент — статические файлы, отдаётся любым веб-сервером.

7.5. Сервер модели

Подробная процедура, включая наложение патчей MTP и переквантование весов, — `backend/docs/MODEL-SERVER.md`. Кратко:

```
llama-server -m GigaChat3.1-10B-A1.8B-q4_K_M.gguf \
  --host 127.0.0.1 --port 8090 -t 6 -c 8192 \
  --spec-type draft-mtp --spec-draft-n-max 1
```

Сервер приложения подключается к нему переменными окружения:

```
ML_PROVIDER=http
ML_BASE_URL=http://127.0.0.1:8090
```

Без них сервер работает на заглушке: интерфейс, классификация и оценка полностью функциональны, отсутствуют только реплики заявителя.

7.6. Проверки

```
cd backend && cargo test            # 138 проверок
cd frontend/e2e && npx playwright install chromium   # один раз
cd frontend && npm test             # 35 сценариев в браузере
```

Тестовый прогон поднимает сервер и клиент самостоятельно.

---

8. Перечень использованных библиотек

Полные версии зафиксированы в `backend/Cargo.lock` и `frontend/apps/react/package-lock.json`.

8.1. Сервер — прямые зависимости

Библиотека	Версия	Лицензия	Назначение
tokio	1.53.1	MIT	асинхронная среда выполнения
axum	0.8.9	MIT	HTTP-сервер и маршрутизация
tower, tower-http	0.5.3, 0.6.11	MIT	промежуточные слои, статика, CORS
sqlx	0.8.6	MIT / Apache-2.0	доступ к PostgreSQL, миграции
serde, serde_json	1.0.229, 1.0.151	MIT / Apache-2.0	сериализация
argon2, password-hash	0.6.0, 0.6.1	MIT / Apache-2.0	хранение паролей
jsonwebtoken	11.1.0	MIT	токены доступа
reqwest	0.12.28	MIT / Apache-2.0	обращение к серверу модели
tokio-tungstenite	0.29.0	MIT	канал событий WebSocket
utoipa, utoipa-swagger-ui	5.5.0, 9.0.2	MIT / Apache-2.0	схема OpenAPI
calamine	0.36.1	MIT	чтение файлов классификатора
chrono	0.4.45	MIT / Apache-2.0	время и нормативы
uuid	1.26.1	Apache-2.0 / MIT	идентификаторы
tracing, tracing-subscriber	0.1.44, 0.3.23	MIT	журналирование
anyhow, thiserror	1.0.104, 2.0.20	MIT / Apache-2.0	обработка ошибок
rand	0.10.3	MIT / Apache-2.0	генерация значений
async-trait	0.1.92	MIT / Apache-2.0	трейты адаптеров моделей
dotenvy	0.15.7	MIT	чтение настроек
futures-util, http-body-util	0.3.34, 0.1.5	MIT	вспомогательные

Полный граф — **344 пакета**. Распределение лицензий: MIT и/или Apache-2.0 — 280, Unicode-3.0 — 18, остальные (BSD, ISC, Zlib, Unlicense, CDLA-Permissive) — 46. **Копилефт-лицензий, обязывающих раскрывать исходный код, в графе нет.**

8.2. Клиент — прямые зависимости

Библиотека	Версия	Лицензия	Назначение
react, react-dom	19.3.0	MIT	интерфейс
react-router-dom	7.18.4	MIT	маршрутизация
zustand	5.0.15	MIT	состояние
vite, @vitejs/plugin-react	8.3.0, 6.1.1	MIT	сборка

Библиотека	Версия	Лицензия	Назначение
sass	1.104.1	MIT	стили
typescript	6.0.3	Apache-2.0	типизация
oxlint	1.83.0	MIT	статический анализ
@types/*	—	MIT	описания типов

Диаграммы аналитики нарисованы средствами SVG — сторонней библиотеки для графиков в поставке нет.

В дереве сборки присутствует `lightningcss` 1.33.0 под **MPL-2.0** (преобразователь CSS внутри Vite). Это инструмент сборки: в поставляемый клиент его код не попадает, изменения в него не вносились, поэтому требования MPL о раскрытии изменённых файлов не возникают.

8.3. Проверки (в поставку не входят)

Playwright (Apache-2.0) — сценарии в браузере.

8.4. Модель

Компонент	Лицензия
GigaChat 3.1 10B-A1.8B	MIT
llama.cpp	MIT

Лицензия MIT у весов модели существенна: она разрешает коммерческое использование, в отличие от распространённых среди русскоязычных моделей лицензий с оговоркой NonCommercial, неприменимых для государственного заказчика.

Два патча к llama.cpp (`backend/docs/mtp-1-converter.patch`, `mtp-2-runtime.patch`) написаны в рамках этой работы и распространяются на условиях лицензии llama.cpp.

---

9. Состав документации репозитория

Файл	О чём
`README.md`	обзор, быстрый старт
`BACKEND.md`	схема базы, контракт API, домен
`INTEGRATION.md`	встраивание в стороннюю систему
`PROVENANCE.md`	происхождение каждого набора данных
`OPEN-QUESTIONS.md`	вопросы к заказчику и принятые допущения
`CONTRIBUTING.md`	правила работы с кодом
`backend/docs/MODEL-SERVER.md`	сборка и запуск сервера модели

Файл	О чём
`frontend/README.md`	архитектура клиента
`frontend/SCENARIOS.md`	сопоставление с экранами боевой системы
`frontend/DEFECTS.md`	найденные проверками дефекты